

Documentatie

Proiect Java

Detectie si Recunoastere Faciala prin SVM

Ștefan-Raul Țuga

M532

Arhitectura Proiectului

Proiectul implementeaza un sistem complet de detectie si recunoastere faciala bazat pe: SVM (Support Vector Machine) cu kernel Sigmoid pentru clasificare; HOG (Histogram of Oriented Gradients) pentru extragerea trasaturilor; SMO (Sequential Minimal Optimization) pentru rezolvarea problemei duale SVM.

Organizarea Pachetelor

Pachet	Descriere
face	Algoritmi de detectie faciala: HOG, SlidingWindow, FaceDetector, ImageUtils
model	Modelul SVM: SVMModel, TrainingData – serializare manuala binara
alg	Algoritmi de invatare: SMO, Perceptron, DualPerceptron, MaxMarginPerceptron, Median
webcam	Camera web si recunoastere: WebcamCapture, FaceRecognizer, DataCollector, ImageViewer
gui	Interfata grafica AWT: Design, Panel_, SimulationControl, Settings, About
io	Structuri de date: Vector, Clasa, Attribute, InputData, OutputData
svm	Clasa principala SVM – punctul de intrare al aplicatiei

Descrierea Claselor

face.HOG

Implementeaza algoritmul Histogram of Oriented Gradients descris de Dalal si Triggs (2005). Extrage un vector de trasaturi de 8100 dimensiuni dintr-o imagine de 128x128 pixeli.

Atribute

Atribut	Tip	Descriere
cellSize	int	Dimensiunea celulei in pixeli
blockSize	int	Dimensiunea blocului in celule
nbins	int	Numarul de directii in histograma orientarilor
NORM_EPS	double	Constanta de stabilitate numerica la normalizarea L2 (10^{-5})

Metode

Metoda	Return	Descriere
extract(int[], int, int)	float[]	Extrage vectorul HOG dintr-o imagine 2D
extract(int[], int, int)	float[]	Suprasarcina pentru array 1D (format BufferedImage)
toGrayscale(...)	float[][]	Converteste RGB la gri: $0.299R + 0.587G + 0.114B$
computeGradients(...)	void	Calculeaza gradientii Gx si Gy cu filtrul centrat [-1, 0, 1]
computeCellHistograms(...)	float[][]	Histograme per celula cu soft binning bilinear pe 9 directii. Variabila interna: histogrameCell
normalizeAndConcatenate(...)	float[]	Normalizare L2-Hys per bloc si concatenare in vectorDescriptor
getVectorSize(int, int)	int	Calculeaza dimensiunea vectorului HOG pentru o rezolutie data

face.ImageUtils

Clasa utilitara cu metode statice pentru procesarea imaginilor.

Metoda	Return	Descriere
load(String)	BufferedImage	Incarca imagine din fisier (JPG, PNG, BMP, GIF)
save(BufferedImage, String)	boolean	Salveaza imagine pe disc in format JPG
resize(BufferedImage, int, int)	BufferedImage	Redimensionare bilineara – medie ponderata a 4 vecini
crop(BufferedImage, int, int, int, int)	BufferedImage	Decupare regiune cu clipping automat la margini
getPixels(BufferedImage)	int[]	Extrage pixelii ca array 1D row-major
loadAndExtractHOG(...)	float[]	Pipeline: incarcare + resize + HOG
cropResizeHOG(...)	float[]	Pipeline: crop + resize + HOG (pentru sliding window)
isImageFile(String)	boolean	Verifica extensia: jpg, jpeg, png, bmp, gif

face.SlidingWindow

Implementeaza detectia prin fereasta glisanta cu piramida de imagini. Parcurge imaginea la mai multe scari si aplica clasificatorul SVM pe fiecare fereasta. Detectiile redundante sunt eliminate prin NMS cu masura IoU.

Atribute

Atribut	Tip	Descriere
winW, winH	int	Dimensiunea ferestrei de detectie (128x128 pixeli)
stepSize	int	Pasul glisarii ferestrei (implicit 48 pixeli)
scaleFactor	double	Factorul de reducere intre scari (implicit 2.0)
nmsThresh	double	Pragul IoU pentru NMS – suprapunere mai mare decat acest prag elimina detectia

Metode

Metoda	Descriere
detect(BufferedImage, Classifier, HOG)	Detectie pe piramida de imagini. Variabile interne: candidatiDetectie (inainte de NMS), detectiiFinale (dupa NMS), esteSuprimat[] (marcheaza detectiile eliminate). Returneaza lista de detectii dupa NMS.
nonMaxSuppression(List)	Algoritmul greedy NMS: sorteaza dupa scor, elimina suprapunerile cu IoU > nmsThresh
calcIoU(Detection a, Detection b)	Calculeaza Intersection over Union intre doua dreptunghiuri
getBest(List)	Returneaza detectia cu scorul maxim
getLargest(List)	Returneaza detectia cu aria maxima (folosita la cerinta 2)

face.FaceDetector

Antreneaza si utilizeaza clasificatorul SVM binar pentru detectia capului.

Atribute

Atribut	Tip	Descriere
hog	HOG	Instanta HOG cu parametrii Dalal & Triggs
sw	SlidingWindow	Fereastra 128x128, stepSize=48, scaleFactor=2.0
model	SVMModel	Modelul SVM antrenat pentru detectia cap/non-cap
DEFAULT_MODEL_PATH	String	Calea implicita pentru salvarea modelului: svm/face_detector.model
DEFAULT_HOG_PATH	String	Calea implicita pentru vectorii HOG: svm/face_detector_hog.dat

Metode

Metoda	Descriere
train(posDir, negDir, negPerImg, modelPath, hogPath)	Antreneaza SVM: extrage HOG din imaginile pozitive si negative, antreneaza SMO (C=1.0, gamma=0.001, coef0=-1.0, maxIter=10000), aplica pruneSuportVectors(0.01), salveaza model si vectori HOG
loadModel(String)	Incarca modelul preantrenat din fisier binar
detectLargestHead(BufferedImage)	Detecteaza toate capetele, returneaza portiunea 128x128 cu aria maxima (cerinta 2)
detectAll(BufferedImage)	Detecteaza toate capetele prin SlidingWindow, returneaza lista completa
isReady()	Returneaza true daca modelul este incarcat si pregatit pentru detectie

alg.SMO

Implementeaza algoritmul SMO cu kernel Sigmoid. Rezolva problema duala de optimizare SVM.

Atribute

Atribut	Tip	Descriere
C	double	Parametrul de regularizare – penalizeaza erorile de clasificare
gamma	double	Scala produsului scalar in kernelul sigmoid (implicit 0.001)
coef0	double	Termenul liber din kernelul sigmoid (implicit -1.0)
tol	double	Toleranta pentru conditiile KKT (implicit 0.001)
maxIter	long	Numarul maxim de iteratii SMO
alpha[]	double[]	Multiplicatorii Lagrange – solutia problemei duale
b	double	Bias-ul (termenul liber) al hiperplanului de separare
errors[]	double[]	Cache pentru erorile de clasificare: errors[i] = f(xi) - yi

Metode

Metoda	Descriere
createStandalone(C, gamma, coef0, tol, maxIter)	Factory method – creeaza SMO fara GUI, pentru antrenare in thread separat
train(Vector[])	Antreneaza SMO: bucla principala alternand examineAll si examinare vectori suport activi pana la convergenta
kernel(float[], float[])	Calculeaza $K(x_i, x_j) = \tanh(\gamma x_i \cdot x_j + \text{coef0})$
examineExample(int, Vector[])	Verifica conditiile KKT, gaseste al doilea multiplicator, optimizeaza perechea (alpha_i, alpha_j)
step(int, int, Vector[])	Optimizeaza o pereche de multiplicatori Lagrange; actualizeaza alpha, b si cache-ul errors[]
score(float[])	Calculeaza $f(x) = \sum(\alpha_i * y_i * K(x_i, x)) + b$ pentru un vector HOG
classify(float[])	Returneaza +1 daca score >= 0, altfel -1

model.SVMModel

Stocheaza parametrii unui clasificator SVM antrenat si implementeaza serializarea manuala prin DataOutputStream/DataInputStream.

Atribute

Atribut	Tip	Descriere
personName	String	Pseudonimul persoanei asociate clasificatorului
alpha[]	double[]	Multiplicatorii Lagrange (copie profunda din SMO prin System.arraycopy)
labels[]	int[]	Etichetele +1/-1 (copie profunda – evita aliasing la re-antrenarea SMO pentru alta persoana)
supportVectors[][]	float[][]	Vectorii suport stocati ca float[][] pentru eficienta
b	double	Termenul liber al hiperplanului de separare
gamma, coef0	double	Parametrii kernelului sigmoid (copiati din SMO)

Metode

Metoda	Descriere
SVMModel(SMO smo, String name)	Constructor: copiaza alpha si labels prin System.arraycopy() pentru a evita suprascrierea datelor la re-antrenarea SMO pentru alta persoana
score(float[])	Calculeaza $f(x) = \sum(\alpha_i * \text{labels}_i * K(\text{sv}_i, x)) + b$
classify(float[])	Returneaza +1 daca score ≥ 0 , altfel -1
pruneSuportVectors(double threshold)	Elimina vectorii suport cu $ \alpha < \text{threshold}$ pentru reducerea memoriei
save(String) / load(String)	Serializare/deserializare manuala binara: dimensiuni, alpha, b, labels, vectori suport
saveAllWithHOG(SVMModel[], TrainingData, String)	Salveaza N modele + vectorii HOG de antrenare intr-un singur fisier binar (cerinta 6)
loadAllWithHOG(String)	Incarca N modele + vectorii HOG; returneaza Object[]{SVMModel[], TrainingData}

model.TrainingData

Stocheaza vectorii HOG si etichetele pentru antrenament cu redimensionare dinamica.

Metoda	Descriere
add(float[], int)	Adauga un exemplu; dubleaza capacitatea daca array-ul este plin
grow()	Dubleaza capacitatea array-urilor si copiaza datele existente
addAll(TrainingData)	Adauga toate exemplele din alt set de date
toVectors()	Converteste la format io.Vector[] pentru SMO
countPositive() / countNegative()	Numara exemplele cu eticheta +1 respectiv -1
save(String) / load(String)	Format binar: [int N][int dim][int*N etichete][float*N*dim vectori]

webcam.WebcamCapture

Captureaza imagini de la camera web folosind OpenCV VideoCapture. Utilizeaza OpenCV exclusiv pentru captura video, conform conditiei 3 din cerinte.

Metoda	Descriere
loadOpenCV()	Incarca biblioteca nativa OpenCV o singura data prin System.loadLibrary
open()	Deschide camera si seteaza rezolutia (640x480) si FPS-ul (10)
close()	Inchide camera si elibereaza resursele OpenCV
captureFrame()	Captureaza un frame si converteste BGR -> RGB in Java pur, fara Imgproc
captureFrameWithDelay()	Captureaza frame si asteapta intervalul corespunzator FPS-ului
matToBufferedImageBGR(Mat)	Conversie Mat CV_8UC3 BGR -> BufferedImage RGB

webcam.DataCollector

Colecteaza imaginile de antrenament de la camera web. Foloseste detectLargestHead() si salveaza imaginile 128x128 cu formatul pseudonim_YYYYMMDD_HHmss_SSS.jpg (cerinta 3).

Atribute

Atribut	Tip	Descriere
detector	FaceDetector	Detectorul SVM pentru localizarea capului in frame
webcam	WebcamCapture	Instanta partajata cu FaceRecognizer – o singura camera deschisa, evita conflictul de acces
rootDir	String	Directorul radacina pentru imaginile de antrenament (implicit training_data/)
imagesPerPerson	int	Numarul de imagini de capturat per persoana
DATE_FORMAT	SimpleDateFormat	Formatul timestamp-ului: yyyyMMdd_HHmss_SSS
stopRequested	volatile bool	Flag thread-safe pentru oprirea colectarii din alt thread

Metode

Metoda	Descriere
collect(String personName)	Buclo principală: captureaza frame, apeleaza detectLargestHead(), salveaza imaginea 128x128 cu numele pseudonim_timestamp.jpg. Variabila interna imageFata: crop-ul 128x128 al capului detectat. Notifica ProgressListener la fiecare imagine salvata.
collectMultiple(String[], long)	Colecteaza pentru mai multe persoane consecutiv cu pauza intre ele
stop()	Seteaza stopRequested=true – oprire thread-safe din exteriorul thread-ului de colectare
countImages(String)	Numara imaginile salvate pentru o persoana in directorul sau
getPersonNames()	Returneaza pseudonimele din subdirectoarele rootDir
setProgressListener(...)	Inregistreaza callback-urile: onProgress, onDone, onNoFaceDetected

webcam.ImageViewer

Permite vizualizarea si stergerea imaginilor de antrenament (cerinta 4). Afiseaza imaginile per persoana cu navigare stanga/dreapta. Stergerea este directa, fara dialog de confirmare, pentru eficienta procesului de curatare a datelor.

webcam.FaceRecognizer

Antreneaza clasificatoarele per persoana si realizeaza recunoasterea faciala live (cerintele 6 si 8). Strategie one-vs-all: imaginile persoanei curente primesc eticheta +1, toate celelalte -1. Desenarea dreptunghiurilor si a textului se face cu Imgproc din OpenCV.

Atribute cheie

Atribut	Tip	Descriere
personModels[]	SVMModel[]	Clasificatoarele per persoana antrenate prin one-vs-all
running	volatile bool	Flag thread-safe pentru oprirea recunoasterii live
MODELS_PATH	String	Calea fisierului cu modele: svm/face_models_hog.dat
CULOARE_CHENAR	Scalar	Culoarea verde BGR (0,255,0) pentru desenare OpenCV
GROSIME_LINIE	int	Grosimea dreptunghiului desenat pe imagine (2 pixeli)
SCALA_TEXT	double	Marimea fontului pentru pseudonimul afisat (0.75)
setDateHOG	TrainingData	Acumuleaza toti vectorii HOG din toate persoanele pentru salvarea la cerinta 6

Metode

Metoda	Descriere
trainAll(String modelsPath)	One-vs-all pentru toate persoanele: extrage HOG din imaginile salvate, construiesc setul (+1/-1), antreneaza SMO, aplica pruneSupportVectors(0.01), salveaza modele + HOG. Variabila interna setDateHOG acumuleaza toti vectorii.
loadModels(String)	Incarca modelele si vectorii HOG din fisierul combinat
startLive()	Recunoastere live la 10 FPS: detectie cap -> dreptunghi verde (Imgproc.rectangle) -> HOG -> scor clasificatori. Afiseaza pseudonimul (Imgproc.putText) daca bestScore > 0. Functioneaza si cu o singura persoana in baza de date.
startLiveAsync()	Pornește recunoasterea live intr-un thread daemon
stopLive()	Oprește recunoasterea prin setarea running = false
getWebcam()	Returneaza instanta WebcamCapture partajata cu DataCollector

Fluxul de Executie

Antrenarea Detectorului de Cap

1. Se incarca imaginile pozitive (fete) din positives/ si se extrage vectorul HOG de 8100 dimensiuni.
2. Se extrag patch-uri aleatoare din imaginile negative (negatives/).
3. Se antreneaza SMO cu kernel Sigmoid (C=1.0, gamma=0.001, coef0=-1.0, maxIter=10000).
4. Se elimina vectorii suport cu contributie neglijabila prin pruneSupportVectors(0.01).
5. Se salveaza modelul in svm/face_detector.model si HOG in svm/face_detector_hog.dat.

Colectarea Imaginilor de Antrenament

1. Se porneste camera web (640x480, 10 FPS) – instanta partajata cu FaceRecognizer.
2. Pentru fiecare frame se apeleaza detectLargestHead().
3. Daca se detecteaza un cap, imaginea 128x128 se salveaza cu formatul pseudonim_YYYYMMDD_HHmmss_SSS.jpg.
4. Se repeta pana la imagesPerPerson imagini sau pana la Stop.

Antrenarea Clasificatoarelor per Persoana

1. Pentru fiecare subdirector din training_data/ (o persoana = un subdirector):
 - . Imaginile persoanei primesc eticheta +1; imaginile tuturor celorlalte persoane primesc eticheta -1.
 - . Se antreneaza SMO (maxIter=1000000, pruneSupportVectors(0.01)).
4. Toate modelele + vectorii HOG sunt salvate intr-un singur fisier: svm/face_models_hog.dat.

Recunoastere Faciala Live

1. Camera web captureaza frame la 10 FPS.
2. detectAll() ruleaza sliding window cu piramida de imagini.
3. Se pastreaza detectia cu scorul maxim (getBest), daca depaseste pragul 0.15.
4. Imgproc.rectangle() deseneaza dreptunghiul verde.
5. Se extrage vectorul HOG din dreptunghiul detectat.
6. Fiecare clasificator evalueaza vectorul HOG; se alege persoana cu scorul maxim pozitiv.
7. Imgproc.putText() scrie pseudonimul deasupra dreptunghiului daca bestScore > 0.
8. Imaginea adnotata este trimisa la GUI prin FrameListener.